BÖLÜM

1

Visual Studio ve Windows Forms Ortamı



BU BÖLÜMÜN SONUNDA

- Konsol uygulaması ile form uygulaması arasındaki farkı açıklayabilecek,
- Visual Studio'da Windows Forms projesi oluşturup çalıştırabilecek,
- Toolbox, Solution Explorer, Properties ve kod/tasarım pencerelerini kullanabilecek,
- Form1.cs dosyasındaki sınıf, kurucu metot ve olay metotlarını tanıyabilecek,
- Programın Main → kurucu → Load → olaylar sırasıyla nasıl çalıştığını izleyebilecek,
- MessageBox ile kullanıcıya mesaj gösterebileceksiniz.

KampüsCepte · öğrenciler için ücretsiz ders notları

Konsolda yazdığımız programlar kullanıcıyla yalnızca metin üzerinden konuşur. Windows Forms ise pencereler, butonlar ve kutularla çalışan, kullanıcının hamlesini bekleyen bir program türüdür. Bu bölümde bu ortamı tanıyor ve ilk form uygulamamızı yazıyoruz.

1.1 Konsoldan Forma: Olay Güdümlü Programlama

Konsol uygulamalarında program **yukarıdan aşağıya** akar: `Main` metodundaki komutlar sırayla çalışır, gerektiğinde `Console.ReadLine()` ile kullanıcıdan veri beklenir ve son satıra gelindiğinde program biter. Programın akışını tamamen programcı belirler.



Her tıklama bir olaydır; olaya bağlı kod o anda çalışır

Windows Forms uygulamasında ise program açıldıktan sonra bir pencere gösterir ve **bekler**. Bundan sonra neyin, ne zaman çalışacağına kullanıcının hamleleri karar verir: bir butona tıklanması, bir kutuya yazı yazılması, listeden bir eleman seçilmesi. Bu hamlelerin her birine **olay (event)**, bu şekilde kurulan programlara da **olay güdümlü (event-driven)** programlar denir. Programcı, her olay gerçekleştiğinde çalışacak kodu ayrı bir metot içine yazar.

	Konsol uygulaması	Windows Forms uygulaması
Arayüz	Siyah ekranda metin	Pencere, buton, metin kutusu, liste...
Akış	Komutlar sırayla çalışır, sonra program biter	Form açılır, kullanıcının yaptığı olaylar beklenir
Ekrana yazma	`Console.WriteLine(...)`	`MessageBox.Show(...)` veya bir kontrolün `Text` özelliği
Veri alma	`Console.ReadLine()`	TextBox, ComboBox gibi kontrollerin özellikleri
Kodun yeri	Çoğunlukla `Main` içinde	Olay metotlarının içinde (ör. `button1_Click`)

1.2 Yeni Bir Windows Forms Projesi Oluşturma

Visual Studio'da yeni bir form uygulaması açmak için şu adımlar izlenir:

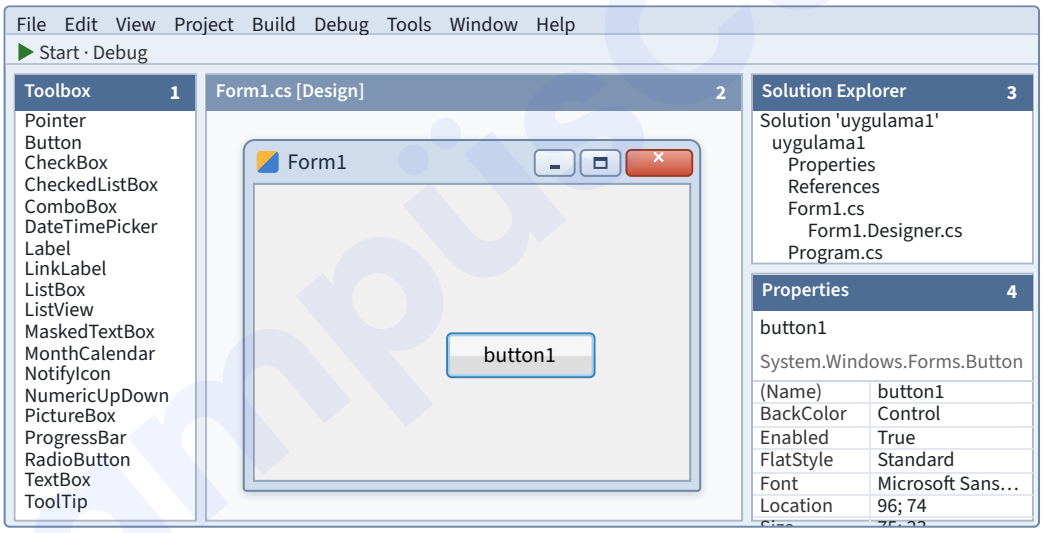
KampüsCepte · öğrenciler için ücretsiz ders notları

1. **File** → **New** → **Project** menüleri sırasıyla tıklanır.
2. Açılan *New Project* penceresinde sol taraftan **Visual C#** → **Windows**, ortadan **Windows Forms Application** seçilir.
3. Alttaki **Name** alanına projenin adı yazılır (ör. `uygulama1`). Ad değiştirilmezse Visual Studio `WindowsFormsApplication1` gibi bir ad verir ve sonraki projelerde sayıyı 2, 3, 4... şeklinde artırır. **Location** alanı projenin kaydedileceği klasörü gösterir.
4. **OK** tuşuna basılır. Ekrana boş bir `Form1` gelir.

> **NOT**
>
> Yeni Visual Studio sürümlerinde (2019, 2022) şablonun adı **Windows Forms App (.NET Framework)** ya da **Windows Forms App** şeklinde görünür. Arama kutusuna *Windows Forms* yazarak şablonu bulabilirsiniz. Dersin örnekleri iki şablonda da aynı şekilde çalışır.

1.3 Visual Studio Pencereleri

Proje açıldığında ekran dört ana bölümden oluşur. Aşağıdaki şekil bu bölümleri, tabloda ise her birinin görevini görüyorsunuz.



Şekil 1.1 *Visual Studio'da bir Windows Forms projesinin çalışma alanı*

No	Pencere	Görevi	View menüsündeki adı
1	**Toolbox**	Forma eklenebilecek kontrollerin (Button, TextBox, Label...) listesi. Kontroller sürüklenip forma bırakılır.	Toolbox
2	**Tasarım penceresi**	Formun görsel olarak düzenlendiği alan. Sekmede `Form1.cs [Design]` yazar.	Designer
3	**Solution Explorer**	Projedeki dosyaları (Form1.cs, Program.cs...) ağaç yapısında gösterir.	Solution Explorer
4	**Properties**	Seçili nesnenin özelliklerini (Text, BackColor, Size...) ve olaylarını listeler.	Properties Window

KampüsCepte · öğrenciler için ücretsiz ders notları

Bu pencerelerden biri yanlışlıkla kapatılırsa **View** menüsünden yeniden açılır. Tasarım ile kod arasında geçiş için **View → Code** (**F7**) ve **View → Designer** (**Shift+F7**) komutları kullanılır; sekmelerdeki `Form1.cs` ve `Form1.cs [Design]` başlıklarına tıklamak da aynı işi görür. Toolbox başlığındaki raptiye simgesi, pencerenin sabit kalmasını ya da otomatik gizlenmesini sağlar.

> **İPUCU**
>
> Kısayol tuşları Visual Studio sürümüne ve seçili ayar profiline göre değişebilir. Örneğin Properties penceresi çoğu sürümde **F4** ile açılır. Kısayolu hatırlamıyorsanız en güvenli yol **View** menüsüdür.

1.4 Programı Çalıştırma

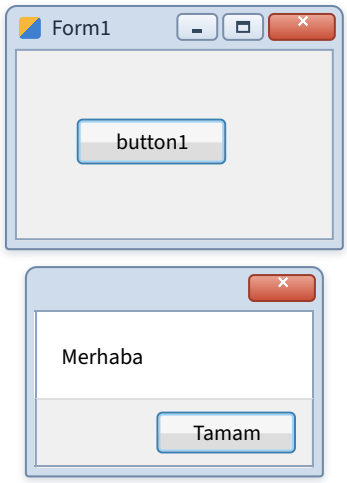
Yazılan program üç yoldan biriyle çalıştırılır: **F5** tuşu, **Debug → Start Debugging** menüsü ya da araç çubuğundaki yeşil ▶ **Start** düğmesi. Üçü de programı hata ayıklama (debug) modunda başlatır. **Ctrl+F5** (*Start Without Debugging*) ise programı hata ayıklayıcı olmadan çalıştırır. Kod derlenemiyorsa program açılmaz ve hatalar **Error List** penceresinde listelenir.

1.5 Form1.cs Kodunun Yapısı

Forma bir buton ekleyip butona çift tıkladığımızda Visual Studio bizi kod penceresine götürür ve tıklama olayı için boş bir metot hazırlar. Metodun içine `MessageBox.Show("Merhaba");` yazdığımızda `Form1.cs` dosyası aşağıdaki gibi görünür.

FORM1.CS

```
1  using System;
2  using System.Collections.Generic;
3  using System.ComponentModel;
4  using System.Data;
5  using System.Drawing;
6  using System.Text;
7  using System.Windows.Forms;
8
9  namespace makine1
10 {
11     public partial class Form1 : Form
12     {
13         public Form1()
14         {
15             InitializeComponent();
16         }
17
18         private void button1_Click(object sender,
   EventArgs e)
19         {
20             MessageBox.Show("Merhaba");
21         }
22     }
23 }
```



KampüsCepte · öğrenciler için ücretsiz ders notları

Bu kısa dosyada nesne tabanlı programlamanın temel kavramlarının neredeyse hepsi vardır:

Satır	Kod	Anlamı
1–7	`using System.Windows.Forms;`	Kullanılacak hazır sınıfların bulunduğu ad alanlarını (namespace) projeye tanıtır. `Form`, `Button`, `MessageBox` gibi sınıflar `System.Windows.Forms` içindedir.
9	`namespace makine1`	Projenin kendi ad alanı. Proje oluşturulurken **Name** alanına yazılan isim buraya gelir.
11	`public partial class Form1 : Form`	**Form1** bir **sınıftır**. İki nokta (`:`) işareti, Form1'in hazır **Form** sınıfından **kalıtım** aldığını gösterir: Form1, bir pencerenin sahip olduğu her şeye (başlık, boyut, renk...) kendiliğinden sahiptir.
11	`partial`	Sınıfın tanımı birden fazla dosyaya bölünmüştür. Tasarımda eklenen kontrollerin kodu Visual Studio tarafından `Form1.Designer.cs` dosyasına yazılır; bizim kodumuz `Form1.cs` dosyasında durur.
13–16	`public Form1() { InitializeComponent(); }`	**Kurucu metot (constructor).** Form1 nesnesi oluşturulurken çalışır. İçindeki `InitializeComponent()`, Designer dosyasında tanımlıdır ve formdaki tüm kontrolleri oluşturup yerleştirir.
18–21	`private void button1_Click(...)`	**Olay metodu.** button1'e tıklandığında çalışır.

NESNE TABANLI BAKIŞ

Dersin adındaki *nesne tabanlı* ifadesi tam olarak burada karşımıza çıkar. `Form1` bir **sınıftır**, yani bir kalıptır. Program çalıştığında bu kalıptan bir **nesne** üretilir ve ekranda gördüğümüz pencere o nesnedir. Forma sürüklediğimiz `button1` de `Button` sınıfından üretilmiş bir nesnedir. Kalıtım sayesinde Form1'e pencere davranışlarını tek satır kod yazmadan kazandırırız. Kalıtımı yazan `: Form` ifadesidir.

1.6 Programın Çalışma Sırası: Main, Kurucu Metot ve Load

Bir form uygulamasının gerçek başlangıç noktası, konsol uygulamalarında olduğu gibi **Main** metodudur. Bu metot Solution Explorer'da görünen `Program.cs` dosyasındadır ve Visual Studio tarafından otomatik yazılır:

PROGRAM.CS

```
using System;
using System.Windows.Forms;

namespace Uygulama1
{
    static class Program
    {
        [STAThread]
        static void Main()
        {
            Application.EnableVisualStyles();
```


KampüsCepte · öğrenciler için ücretsiz ders notları

```
            Application.SetCompatibleTextRenderingDefault(false);
            Application.Run(new Form1());   // Form1 nesnesi oluşturulur ve ekrana gelir
        }
    }
}
```

`Application.Run(new Form1())` satırı önce `new` ile bir Form1 nesnesi oluşturur (bu sırada kurucu metot çalışır), sonra bu formu ekrana getirip uygulamanın **mesaj döngüsünü** başlatır. Form ekrana gelmeden hemen önce formun **Load** olayı tetiklenir. Form kapandığında `Application.Run` sona erer ve program biter.



Şekil 1.2 *Bir Windows Forms uygulamasında kodların çalışma sırası*

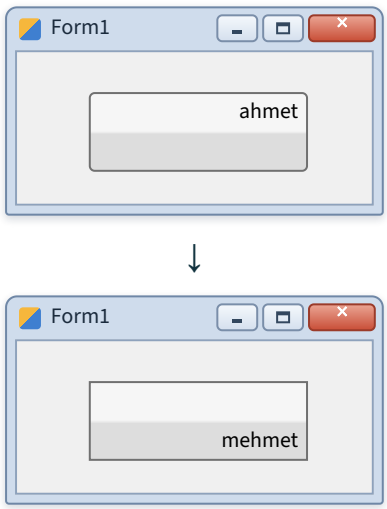
> **DİKKAT**
>
> Bazı kaynaklarda `Form1_Load` için "ana metot gibidir" denir. Bu benzetme yalnızca, Load olayına yazılan kodların kullanıcı hiçbir şey yapmadan, form açılırken **kendiliğinden** çalıştığını anlatmak içindir. Programın başlangıç noktası `Main`'dir ve **kurucu metot Load'dan önce çalışır**.

Load olayına ulaşmak için tasarım penceresinde formun boş bir yerine çift tıklamak yeterlidir. Açılış değerleri (kutulara varsayılan değer yazmak, listeleri doldurmak gibi) genellikle buraya yazılır. Aşağıdaki örnekte form açıldığında buton yazısı **ahmet** olur ve sağ üst köşeye hizalanır. Butona tıklandığında ise yazı **mehmet** olur ve sağ alta kayar.

FORM1.CS — LOAD VE CLİCK OLAYLARI

```
private void Form1_Load(object sender, EventArgs e)
{
    button1.Text = "ahmet";
    button1.TextAlign = ContentAlignment.TopRight;
}

private void button1_Click(object sender, EventArgs e)
{
    button1.Text = "mehmet";
    button1.TextAlign = ContentAlignment.BottomRight;
    button1.FlatStyle = FlatStyle.System;
}
```



1.7 Yazım Kuralları ve Açıklama Satırları

- C# **büyük-küçük harf duyarlıdır**. `MessageBox` ile `messagebox`, `Text` ile `text` farklı adlardır. Yanlış yazılan ad derleme hatasına yol açar.
- Her komut **noktalı virgül** (`;`) ile biter. Kod blokları **süslü parantez** (`{ }`) ile açılıp kapatılır.

KampüsCepte · öğrenciler için ücretsiz ders notları

- Metinler **çift tırnak** içinde yazılır: `"Merhaba"`. Tırnak konmazsa derleyici bunu bir değişken adı sanar.
- **Açıklama satırları** derleyici tarafından dikkate alınmaz. Tek satır için `//`, birden çok satır için `/* … */` kullanılır. Visual Studio açıklamaları yeşil renkte gösterir.

AÇIKLAMA SATIRLARI

```
private void button1_Click(object sender, EventArgs e)
{
    MessageBox.Show("DENEME");

    // Tek satırlık açıklama: bugün hava çok güzel

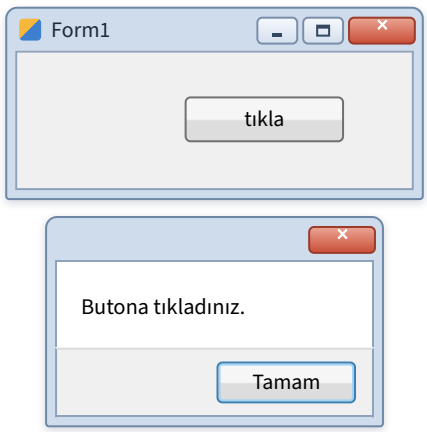
    /* Çok satırlı açıklama:
       Bu satırlar derleyici tarafından
       dikkate alınmaz. */
}
```

Açıklamalar, kodu daha sonra okuyacak kişiye (çoğu zaman bir süre sonraki kendimize) ne yapıldığını anlatmak için yazılır. Ayrıca bir kod parçasını silmeden geçici olarak devre dışı bırakmak için de kullanılır.

1.8 Ekrana Mesaj Gönderme: MessageBox

Konsolda kullandığımız `Console.Write` ve `Console.WriteLine` komutları form ortamında kullanılmaz. Kullanıcıya bir mesaj göstermek için `MessageBox.Show("mesaj")` komutu yazılır. Komut, içinde mesaj ve **Tamam** düğmesi bulunan küçük bir pencere açar. Bu pencere kapatılmadan forma dönülemez.

```
private void button1_Click(object sender, EventArgs
e)
{
    MessageBox.Show("Butona tıkladınız.");
}
```



İPUCU

`MessageBox.Show` yalnızca metin (string) kabul eder. Bir sayıyı göstermek için onu metne çevirmek gerekir: `MessageBox.Show(sonuc.ToString());` ya da metinle birleştirerek `MessageBox.Show("Sonuç: " + sonuc);`. Bu konu 3. bölümde ayrıntılı işlenecektir.

KampüsCepte · öğrenciler için ücretsiz ders notları

1.9 Olaylar ve Olay Metotları

Tasarım penceresinde bir kontrole çift tıklandığında Visual Studio o kontrolün **varsayılan olayı** için boş bir metot oluşturur. En sık kullanılan tetikleyici kontrol butondur ve varsayılan olayı **Click**'tir:

```
private void btnTikla_Click(object sender, EventArgs e)
{
    // Butona tıklandığında buradaki kodlar çalışır.
}
```

Metodun adı *kontrolAdı_OlayAdı* biçimindedir. Parantez içindeki iki parametre her olay metodunda bulunur: `sender` olayı tetikleyen nesneyi, `e` ise olayla ilgili ek bilgileri taşır. Kontrollerin diğer olaylarına **Properties** penceresindeki **şimşek** simgesine (Events) tıklanarak ulaşılır.

Kontrol	Varsayılan olay	Ne zaman tetiklenir?
Form	`Load`	Form ilk kez ekrana gelmeden hemen önce
Button	`Click`	Butona tıklandığında
TextBox	`TextChanged`	Kutudaki metin her değiştiğinde
CheckBox, RadioButton	`CheckedChanged`	İşaret durumu değiştiğinde
ComboBox, ListBox	`SelectedIndexChanged`	Seçili eleman değiştiğinde
DateTimePicker	`ValueChanged`	Seçili tarih değiştiğinde
LinkLabel	`LinkClicked`	Bağlantıya tıklandığında

DİKKAT

Olay metodunun çalışması için metodun olaya **bağlanmış** olması gerekir. Çift tıklama ile oluşturulan metotlarda Visual Studio bu bağlantıyı `Form1.Designer.cs` dosyasına kendisi yazar:

```
// Form1.Designer.cs dosyasında Visual Studio'nun otomatik yazdığı satırlar:
this.Load += new System.EventHandler(this.Form1_Load);
this.button1.Click += new System.EventHandler(this.button1_Click);
```

Bir metodu başka bir projeden kopyalayıp yapıştırmak bu bağlantıyı oluşturmaz. Metot yazılmış görünür ama hiçbir zaman çalışmaz. Böyle bir durumda metot, Properties → Events listesinden ilgili olaya seçilerek bağlanır.

KampüsCepte · öğrenciler için ücretsiz ders notları

NESNE TABANLI BAKIŞ

Bir olay metodu, olayı tetikleyen nesneye `sender` parametresi üzerinden ulaşabilir. `sender` genel `object` türündedir. Nesnenin bir buton olduğunu biliyorsak tür dönüşümü yaparak onun özelliklerini okuyabiliriz:

```
private void button1_Click(object sender, EventArgs e)
{
    Button tiklanan = (Button)sender;   // olayı tetikleyen nesne
    MessageBox.Show(tiklanan.Text + " butonuna tıklandı.");
}
```

Aynı metot birden çok butonun Click olayına bağlanırsa, her tıklamada hangi butona basıldığı bu yöntemle anlaşılır. Bu kullanımı 2. bölümde kod ile buton oluştururken tekrar göreceğiz.

ÖZET

- Windows Forms uygulamaları **olay güdümlüdür**. Kod, kullanıcının yaptığı olaylara bağlı metotların içine yazılır.
- Proje **File → New → Project → Visual C# → Windows → Windows Forms Application** yoluyla oluşturulur. Program **F5** ile çalıştırılır.
- Toolbox, Solution Explorer ve Properties pencereleri kapanırsa **View** menüsünden yeniden açılır. Kod/tasarım geçişi **F7** / **Shift+F7** ile yapılır.
- `Form1` bir sınıftır ve `: Form` ifadesiyle Form sınıfından kalıtım alır. `partial` ifadesi, sınıfın Form1.cs ile Form1.Designer.cs arasında bölündüğünü gösterir.
- Çalışma sırası: **Main → new Form1() (kurucu, InitializeComponent) → Load → form görünür → kullanıcı olayları**.
- C# büyük-küçük harf duyarlıdır. Komutlar `;` ile biter. Açıklamalar `//` ve `/* */` ile yazılır.
- Form ortamında ekrana mesaj `MessageBox.Show("...")` ile verilir.

KampüsCepte